3D Turbo Racer

CG Edition

A 3D racing game in C++17 / OpenGL with an integrated Computer Graphics Learning Mode



Project Report

Author: Sami Khan
LinkedIn: linkedin.com/in/samikhan07 | GitHub: github.com/samikhan7507

Course: Computer Graphics | Submission: College project

Abstract

3D Turbo Racer - CG Edition is an arcade racing game whose purpose is to make Computer Graphics theory visible. The game is implemented in modern C++17 with fixed-function OpenGL (GLUT). All geometric transformations, the camera and the projection are produced by a hand-written matrix library (`math3d.hpp`) rather than by OpenGL helper calls, and the heads-up display is drawn with classic raster algorithms: a Bresenham speedometer and a minimap drawn with DDA after Cohen-Sutherland clipping. A built-in Learning Mode with eleven interactive pages demonstrates translation, rotation, scaling, reflection, shearing, composite matrices, DDA, Bresenham, clipping, projection and animation using the same code the game uses, each with formulas, live values and a note about where the concept is used in the game. A JavaScript port provides a browser/phone version suitable for free hosting on GitHub Pages. This report describes the design, implementation, testing and the honest limitations of the work.

1. Introduction and Objectives

Computer Graphics courses introduce matrices, rasterisation and clipping in isolation; students rarely see them working together in a real-time program. The objectives of this project were:

• build a playable 3D racing game with curved highways, sharp turns, mountain roads, rough terrain, traffic, obstacles, realistic steering, collision detection, nitro boosts, upgrades, scoring and checkpoints;
• provide four difficulty levels with distinct environments and weather;
• add a Learning Mode that is directly connected to the game code;
• keep C/C++ as the main implementation, while offering a free online/phone version;
• deliver tested, documented, easy-to-build source code.

2. System Architecture

The program is split so that game rules can be tested without a window:

File	Responsibility
math3d.hpp	Vec2/3/4, Mat3 (2D homogeneous) and Mat4 (3D): translate, rotate, scale, reflect, shear, perspective, ortho, lookAt, multiply, toGL (transposition for OpenGL).
cg_algorithms.hpp	DDA, Bresenham (general all-octant and textbook first-octant form with decision parameters), Cohen-Sutherland outcodes and clipping.
game.hpp	Track generator, vehicle physics, traffic, obstacles, pickups, SAT collision, scoring, checkpoints, upgrades/save file, difficulty table, autopilot. No OpenGL.
gfx.hpp	Text, 2D primitives and 3D solids on top of GLUT (avoids freeglut-only calls so macOS builds).
learn_mode.hpp	Eleven Learning Mode pages.
main.cpp	Rendering, HUD, menus, input, state machine, scripted screenshot mode.
tests/selftest.cpp	Headless unit and simulation tests.

Rendering pipeline. Each frame the game computes `P = Mat4::perspective(fov, aspect, n, f)` and `V = Mat4::lookAt(eye, centre, up)`, loads them with `glLoadMatrixf`, and multiplies a per-object model matrix such as `T(pos) * Ry(yaw) * Rx(-pitch)` before drawing. The HUD uses an orthographic 1280x720 projection, letter-boxed to 16:9 so it is correct on any window size.

3. Game Design and Implementation

3.1 Track generation

The road is a polyline of points every 3 m. The generator walks along it integrating heading, *heading += curvature * step*, position and elevation, choosing sections at random from five types: straight, gentle curve, sharp turn, hairpin and S-curve. Curvature is multiplied by a smooth envelope to avoid sudden jumps, and the section type decides terrain: normal highway, mountain (one side rises as a wall, the other drops away) or rough terrain (bumpy gravel, drag and no guard rails). Side terrain is a value-noise height field blended in with distance from the road, giving hills and rough ground.

3.2 Vehicle physics and steering

The car uses a bicycle model with a speed-sensitive steering limit. The maximum steering angle is limited so lateral acceleration $v^2 \cdot \tan(\delta)/L$ cannot exceed the available grip, which depends on tyres, handling upgrades, surface and weather. Velocity is stored as a world vector and decomposed into forward and sideways components every step; the sideways component is damped exponentially at a grip-dependent rate, so low grip (snow, handbrake, gravel) produces visible drifting. Engine force, quadratic drag, rolling resistance, braking, reverse, off-road drag and nitro are applied to the forward component. Physics runs at a fixed 120 Hz sub-step.

3.3 Collision detection and damage

The player car, traffic vehicles and solid obstacles are oriented rectangles tested with the Separating Axis Theorem; the axis of least penetration gives the push-out direction and collision normal. Damage depends on closing speed; contacts have a cool-down so that being pinned does not repeatedly damage the car. Guard rails and boundary posts are modelled as a lateral limit with a bounce. Cones and barrels are destroyed on impact, rocks and barriers are solid.

3.4 Traffic, obstacles, pickups, scoring

Traffic cars and trucks cruise in lanes at difficulty-dependent speeds. Passing a car closely at high relative speed awards a near-miss and builds a combo (score = 50 x combo, plus a little nitro). Blue pickups refill nitro. Checkpoint arches every 700 m add time; finishing awards a bonus proportional to the remaining time. Money earned after each race (score / 12 x difficulty reward) buys upgrades in the Garage; progress is saved to a text file.

Level	Environment	Weather	Length	Grip	Time	Reward
Easy	Sunny Coastal Highway	Clear	3.2 km	100%	60 s + 20/cp	x1.0
Medium	Misty Mountain Pass	Rain + fog	4.2 km	88%	70 s + 22/cp	x1.4
Hard	Night Storm Canyon	Heavy rain, lightning, night	5.2 km	82%	75 s + 28/cp	x1.9
Extreme	Arctic Blizzard Ridge	Snow blizzard	6.0 km	68%	80 s + 30/cp	x2.6

Table 1 - Difficulty levels. Fog distance, traffic density, obstacle density, curve sharpness, slope and rough-terrain probability also scale with difficulty.



Figure 1 - Medium: rain, fog and a mountain wall on the left.



Figure 2 - Hard: night storm with headlight cone; lightning flashes the sky.



Figure 3 - Extreme: blizzard, low grip, snow-capped pines.

4. How Computer Graphics Concepts Are Used in the Game

Concept	Use in the running game
Translation	Placement of every car, obstacle, tree and pickup (Mat4::translate); camera follows by translating the world.
Rotation	Yaw/pitch of cars (rotateY, rotateX), steering wheels, minimap orientation.
Scaling	Truck/vehicle sizes, flame length, speed-dependent FOV scale.
Reflection	Left wheels and mirror are the right-hand ones drawn with Mat4::reflectX() = scale(-1,1,1).
Shearing	Body roll in corners: Mat4::shear applied to body and cabin, wheels stay planted.
Composite matrices	Model = T*Ry*Rx; MVP = P*V*M; minimap = T*S*R*T^-1.
Projection	Perspective projection with FOV 60 deg + 0.28*speed (+12 deg nitro); minimap as top-down projection.
DDA	Minimap road segments rasterised pixel by pixel.
Bresenham	Speedometer needle and ticks rasterised with integer arithmetic.
Clipping	Cohen-Sutherland clips minimap segments to the minimap rectangle before DDA.
Animation	dt-based camera easing, wheel rotation, flame flicker, pickup spin, traffic motion, double buffering.



Figure 4 - Pressing G shows the live model matrix, FOV, projection element and the list of algorithms used by the HUD.

5. CG Learning Mode

The Learning Mode is reachable from the main menu, the pause menu or by pressing L in a race. Every page has a visual canvas, adjustable parameters (or auto-animation), the formula, live numeric values (matrices, outcodes, decision parameters) and a "where this is used in the game" explanation. Because the pages call `Mat3`, `Mat4`, `ddaLine`, `bresenhamTextbook` and `cohenSutherland` from the same headers as the game, the demonstrations cannot drift away from the real implementation.

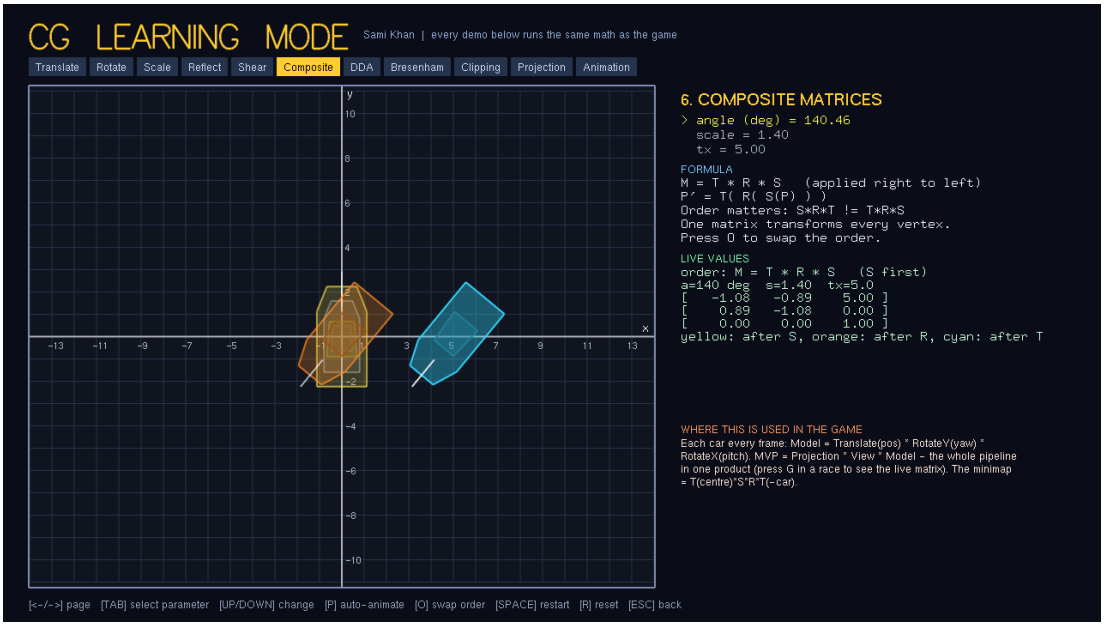


Figure 5 - Composite transformations: ghosts show each step; order can be swapped to show T*R*S differs from S*R*T.

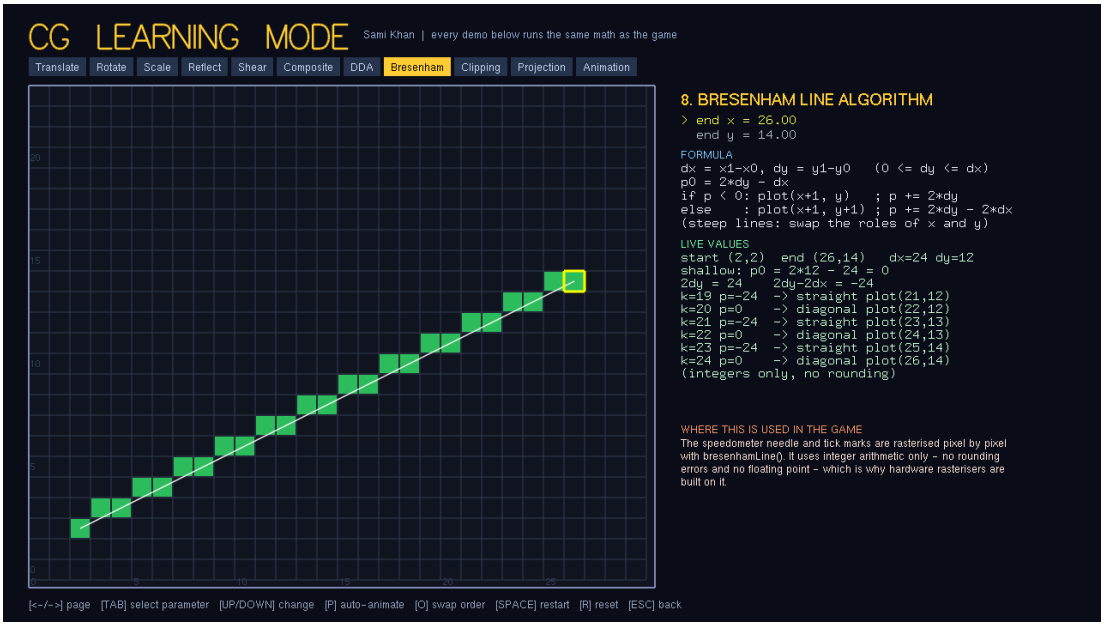


Figure 6 - Bresenham: pixels appear one by one with the decision parameter p_k and the straight / diagonal choice.

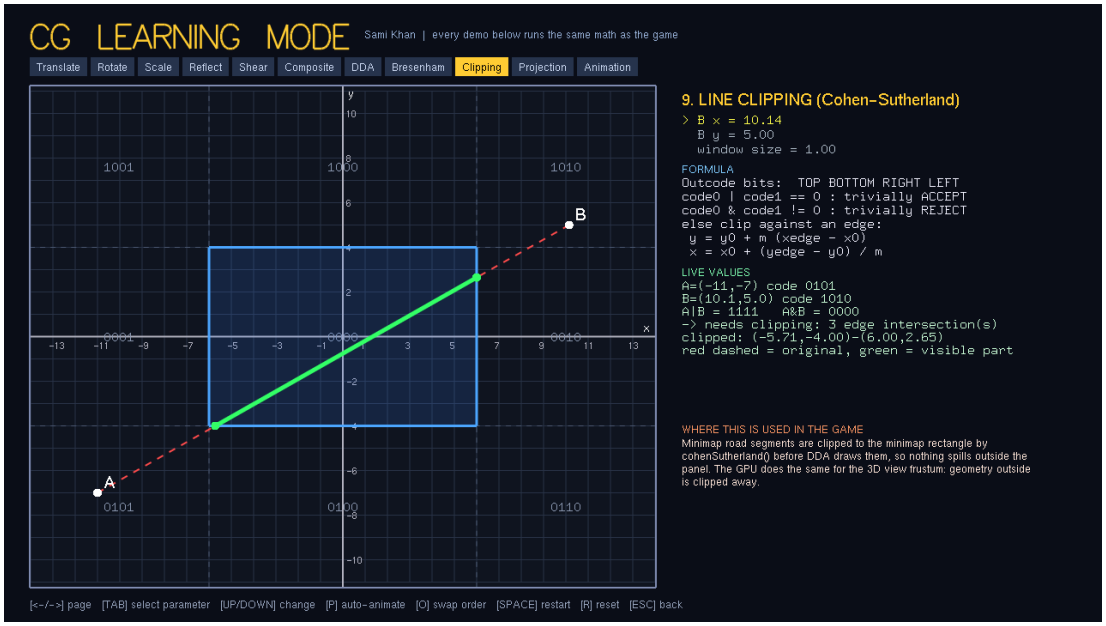


Figure 7 - Cohen-Sutherland: nine regions with outcodes, original line (red, dashed) and clipped segment (green).

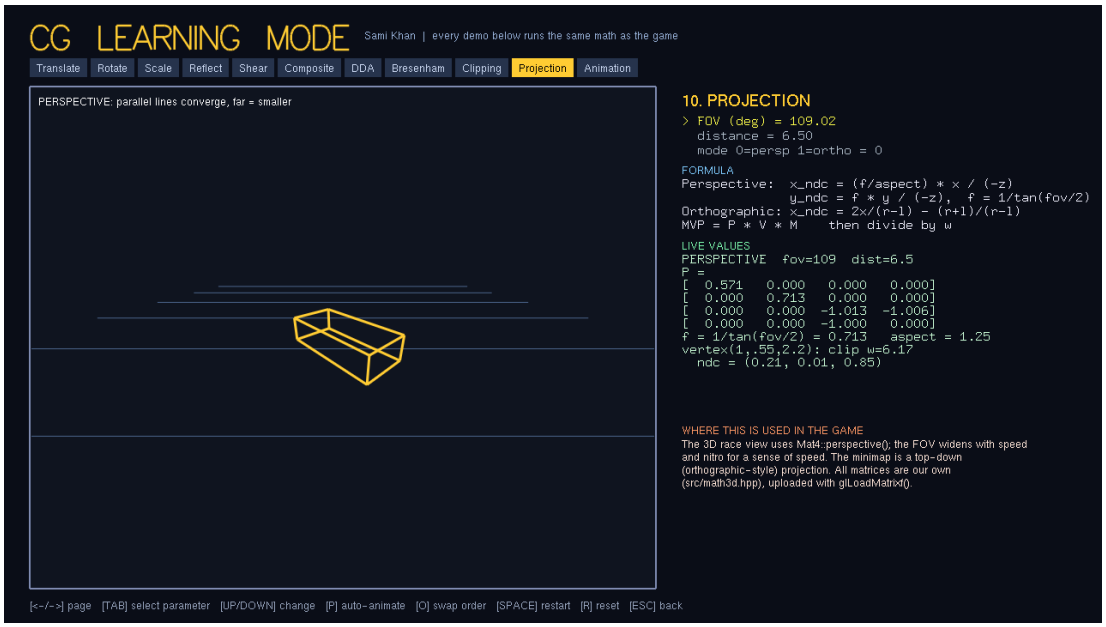


Figure 8 - Projection: wire-frame car box and ground grid with the live perspective matrix (grid lines are clipped with the project's own clipper).

6. User Interface and Screens



Figure 9 - Main menu.



Figure 10 - Difficulty selection with environment, weather and best score.

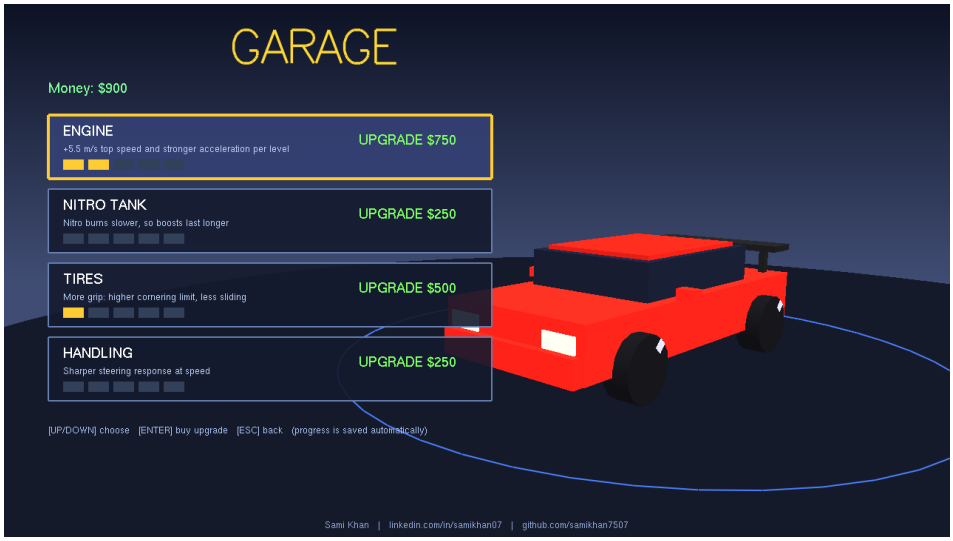


Figure 11 - Garage with four five-level upgrades.



Figure 12 - Result screen with statistics, money, best score and stars.

7. Web / Phone Version

A single HTML file (`web/index.html`) re-implements the game in JavaScript for browsers and phones: the same difficulty table and physics constants, a seeded track generator following the same scheme, SAT collisions, nitro, upgrades (stored in localStorage), checkpoints and the eleven Learning Mode pages. It projects all 3D points itself onto a 2D canvas, which keeps it dependency-free. It adds touch controls (steer, brake, nitro, automatic throttle). A GitHub Actions workflow deploys the folder to GitHub Pages. **This is a re-implementation, not the C++ code compiled to WebAssembly**; the Emscripten toolchain was not available in the build environment, so a WebAssembly build was not attempted.



Figure 13 - Web version: phone race with touch buttons, phone menu, desktop browser (Hard).

8. Testing and Results

Headless tests (C++). `selftest` runs 35 checks, all passing. They cover: R(30°)R(45°)=R(75°); composite equals sequential application; non-commutativity; reflection, shear, translation results; perspective sends points at the near and far planes to NDC -1 and +1; lookAt; column-major export; DDA and Bresenham agreement within one pixel on 2,000 random lines; textbook p_k Bresenham identical to the general version on 500 random lines; Cohen-Sutherland accept/reject cases and agreement with brute-force sampling on 3,000 random segments; head-on collision registers damage; nitro and engine upgrades raise speed.

Simulated races. A built-in autopilot drove all four difficulties (60 Hz simulation). Representative results:

Level	Result	Sim. time	Score	Checkpoints	Hits	Damage
Easy	Finished	119 s	5788	4/4	8	92%
Medium	Time up	183 s	4329	5/6	5	75%
Hard	Wrecked	237 s	5137	6/7	2	100%
Extreme	Wrecked	143 s	2566	3/8	6	100%

The autopilot is deliberately simple and cautious; it is a regression and smoke test, not a measure of human play. It was used to find and fix real defects: repeated per-frame damage while pinned against an obstacle, results not being written back to the saved profile, and time limits that were too tight. Easy is finished by the autopilot; the harder levels are demanding and were **not** tuned with human testers.

End-to-end run. The complete application (menus, four races with weather, garage, eleven Learning pages, result screen) was executed in the scripted `--screenshots` mode under Xvfb with Mesa software OpenGL; 2,600+ frames rendered in about one minute with no crashes, and 21 screenshots were captured and inspected. **Web:** run in headless Chromium at 1280x720 and 390x780 with touch emulation; zero page or console errors; all four difficulties and all eleven learning pages rendered.

9. Platform Limitations (honest summary)

• **Tested platform:** Linux (Ubuntu 24.04, g++ 13, freeglut, Mesa software renderer). Windows and macOS build instructions are provided but untested; real-GPU frame rate was not measured.
• **Phones:** the C++ game cannot run on phones. The JavaScript port was verified only through Chromium emulation, not on physical devices.
• **Fidelity:** the web port has different random tracks and simpler car models, and lacks mirrored wheels and lit surfaces.
• **Legacy OpenGL:** fixed-function pipeline via GLUT was chosen for portability and clarity; Apple deprecates GLUT/OpenGL.
• **No audio**, no gamepad support, no network multiplayer.
• **Balance:** difficulty tuning is based on an autopilot, not human playtests. Solid rocks can pin a car (reverse to escape).
• **Publishing:** the GitHub Pages workflow is included but must be run from the author's own repository.

10. Conclusion and Future Work

The project delivers a complete, tested C++/OpenGL racing game in which the core CG topics are not only taught but used: the matrices that place and project every object, the algorithms that draw the HUD, and the animation loop that drives the scene. Future work includes compiling the C++ sources to WebAssembly with Emscripten so the web build shares the desktop code, moving to modern

shader-based OpenGL with real lighting and shadows, adding sound, gamepad and touch support to the desktop build, human playtesting to tune difficulty, ghost laps and online leaderboards.

References

[1] D. Hearn, M. P. Baker, *Computer Graphics with OpenGL*, Pearson.
[2] J. E. Bresenham, "Algorithm for computer control of a digital plotter", *IBM Systems Journal*, 4(1), 1965.
[3] I. Sutherland, "Sketchpad", MIT, 1963 (clipping origins); Cohen-Sutherland line clipping.
[4] Khronos Group, OpenGL 2.1 specification; freeglut documentation (freeglut.sourceforge.net).
[5] G. van den Bergen, *Collision Detection in Interactive 3D Environments* (Separating Axis Theorem).
[6] M. Pacejka, *Tire and Vehicle Dynamics* (background on lateral grip limits; simplified in this project).

Appendix A - Build and Run

```
sudo apt install build-essential cmake freeglut3-dev libglu1-mesa-dev
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j
./build/turbo_racer    # play
./build/selftest     # headless tests
```

Controls: W/S accelerate/brake, A/D steer, SPACE nitro, B handbrake, C camera, G CG overlay, L Learning Mode, P pause, R restart, F fullscreen.